

Документация API Kvant Server

Версия: 1.4
Дата: 2026-05-20
Протокол: HTTPS

Оглавление

- 1. Общие сведения
- 2. Авторизация
- 3. Auth — Авторизация
- 4. Employees — Сотрудники
- 5. Events (org) — Мероприятия организации
 - Документы к мероприятиям организации
- 6. Events (part) — Мероприятия участия
 - Документы к мероприятиям участия
 - Ученики мероприятий участия
- 7. Schedule — Расписание
- 8. Reference — Справочники
- 9. Rent — Аренда помещений
- 10. Students — Студенты
- 11. Attendance — Посещаемость
- 12. Groups — Группы
- 13. Desktop Updates — Обновления клиента
- 14. Коды ошибок и пагинация

1. Общие сведения

Базовый URL

```
https://localhost:3000
```

Порт может отличаться в зависимости от настройки (по умолчанию `3000`). При развёртывании на сервере замените `localhost` на IP или домен.

Формат ответов

Успех	Ошибка
<code>{ success: true, data }</code>	<code>{ success: false, error }</code>
В части эндпоинтов к ответу добавляются поля <code>page</code> , <code>limit</code> , <code>total</code>	Сообщение об ошибке в поле <code>error</code>

Content-Type

- Запрос:** `application/json` (за исключением эндпоинтов загрузки файлов — `multipart/form-data`, см. §5–6, документы к мероприятиям)

- **Ответ:** `application/json` (за исключением скачивания файлов — бинарное тело с `Content-Disposition: attachment`)
-

2. Авторизация

Все эндпоинты, **кроме** `/api/auth/`, требуют авторизации.

Способы передачи токена

1. **Cookie** — `access_token` (устанавливается при логине, `httpOnly, secure`)
2. **Заголовок** — `Authorization: Bearer <accessToken>`

Истечение токенов

- **Access Token** — 1 час
- **Refresh Token** — 7 дней

При истечении access-токена используйте `/api/auth/refresh` с refresh-токеном для получения нового access-токена.

3. Auth — Авторизация

Базовый путь: `/api/auth`

Токен не требуется.

POST `/api/auth/login`

Вход по логину и паролю.

Тело запроса:

```
{
  "login": "string",
  "password": "string"
}
```

Ответ 200:

```
{
  "success": true,
  "data": {
    "accessToken": "string",
    "refreshToken": "string",
    "user": {
      "id": 1,
      "accessLevel": 1
    }
  }
}
```

Access-токен также устанавливается в cookie `access_token`.

POST /api/auth/refresh

Обновить access-токен по refresh-токену.

Тело запроса:

```
{
  "refreshToken": "string"
}
```

Ответ 200:

```
{
  "success": true,
  "data": {
    "accessToken": "string"
  }
}
```

POST /api/auth/logout

Выход, отзыв refresh-токена.

Тело запроса (опционально):

```
{
  "refreshToken": "string"
}
```

Ответ 200:

```
{
  "success": true,
  "data": {
    "ok": true
  }
}
```

POST /api/auth/change-password

Смена пароля текущего пользователя. **Токен обязателен** (cookie `access_token` или `Authorization: Bearer`).

Тело запроса:

```
{
  "old_password": "string",
  "new_password": "string"
}
```

`new_password` — минимум 6 символов.

Ответ 200:

```
{ "success": true, "data": { "ok": true } }
```

Ошибки:

Код	Описание
400	Не указаны поля или пароль короче 6 символов
401	Нет или невалидный access-токен
403	Неверный old_password

4. Employees — Сотрудники

Базовый путь: /api/employees

Токен требуется.

Сотрудники: списки, поиск, KPI, контакты, размеры, назначение на мероприятия.

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	/api/employees	Список активных с должностью	Массив: [{ id_employees, first_name, second_name, patronymic, position, position_name }]
GET	/api/employees/all	Полные поля сотрудников (только активные)	Массив: [{ id_employees, first_name, second_name, patronymic, date_of_birth, position, contact, size, education, gender }]
GET	/api/employees/with-inactive	Все сотрудники, включая неактивных	Массив: [{ id_employees, first_name, second_name, patronymic, position, position_name, is_active }]
GET	/api/employees/schedule	Расписание: занятия, комнаты, дни	Массив: [{ id_employees, id_schedule, room_id, employee_name, day_name, startTime, endTime, room_name }]
GET	/api/employees/short-list	Краткий список id и имя	Массив: [{ id, name }]
GET	/api/employees/sizes	id, имя, пол, размер	Массив: [{ id, name, gender, size }]
GET	/api/employees/search	Поиск по букве	Массив: [{ id_employees, name }]
GET	/api/employees/search/:letter	То же, буква в пути	то же
GET	/api/employees/kpi/:id	KPI сотрудника	Объект: { KPI }
GET	/api/employees/:id	Один сотрудник	Объект: { id_employees, first_name, second_name, patronymic, date_of_birth, position, position_name, contact, size, education, gender, KPI, is_active }

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/employees	Назначить на мероприятие	event_id, employee_id (number)	{ success: true, data: { message } } (201)

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/employees/add	Добавить сотрудника	Обязательно: first_name , second_name , patronymic , date_of_birth , position , login , password , access_level_id . Опц.: contact , size , education , gender , KPI	{ success: true, data: { id } } (201); при занятом логине — 409 (success: false , error)

PUT-эндпоинты

Ответ 200 для всех: { success: true, data: { ok: true } } .

Метод	Путь	Описание	Тело запроса
PUT	/api/employees/kpi	Установить KPI	id , KPI (string)
PUT	/api/employees/contact	Обновить контакт	id , contact
PUT	/api/employees/size	Обновить размер	id , size
PUT	/api/employees/:id	Обновить все данные сотрудника	см. ниже

PUT /api/employees/:id

Обновляет все поля сотрудника.

Тело запроса (application/json):

Поле	Тип	Обязательное	Описание
first_name	string	да	Имя
second_name	string	да	Фамилия
date_of_birth	string	да	Дата рождения (YYYY-MM-DD)
patronymic	string	нет	Отчество
position	number	нет	id должности (null — без должности)
contact	string	нет	Контакт
size	string	нет	Размер
education	string	нет	Образование
gender	string	нет	Пол
KPI	string	нет	KPI
is_active	boolean/number	нет	Если передан — обновляет статус активности (1/true — активен, 0/false — неактивен)

Ответ: { success: true, data: { ok: true } }

Ошибки:

Код	Описание
400	Не указаны обязательные поля или некорректный position
404	Сотрудник не найден

DELETE-эндпоинты

Метод	Путь	Описание	Ответ
DELETE	/api/employees/:id	Удалить сотрудника	{ success: true, data: { ok: true } }

DELETE /api/employees/:id

Полностью удаляет сотрудника из базы. Триггер `employees_BEFORE_DELETE` автоматически каскадно удаляет связанные записи:

- `profile` (→ `refresh_tokens` через триггер `profile_BEFORE_DELETE`)
- `employees_schedule`
- `responsible_for_org_events`
- `responsible_for_part_events`

Ошибки:

Код	Описание
400	Некорректный id
404	Сотрудник не найден

5. Events (org) — Мероприятия организации

Базовый путь: `/api/events/org`

Токен требуется.

Мероприятия типа «организация»: список с фильтрами и пагинацией, полная информация, ответственные, уведомления, CRUD.

Фильтр списка и count по типу мероприятия (org)

Чтобы оставить в выборке только мероприятия заданного **типа организации**, в теле запросов `POST /api/events/org/list` и `POST /api/events/org/count` в объекте `filters` передайте поле `type`: положительное целое число — тот же id, что в справочнике `types_of_organization` (список с названиями: `GET /api/reference/types-of-organization`). Сочетается с остальными полями `filters` (период, даты, поиск, ответственный).

Пример:

```
"filters": {
  "type": 1,
  "period": "three_months"
}
```

Полная таблица всех полей `filters` — в подразделе [Org list and count filters](#).

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	<code>/api/events/org/resp-table</code>	Таблица ответственных	Массив (<code>responsible_for_org_events</code>)
GET	<code>/api/events/org/full-inf/:id</code>	Полная информация о мероприятии	Объект: { <code>id</code> , <code>name</code> , <code>form_of_holding</code> , <code>dates_of_event</code> , <code>day_of_the_week</code> , <code>amount_of_applications</code> , <code>amount_of_planning_application</code> , <code>annotation</code> , <code>result</code> , <code>type</code> , <code>link</code> }. Поле <code>form_of_holding</code> — строка (в БД <code>varchar</code>), не id справочника. Поле <code>type</code> — число (id из <code>types_of_organization</code>); названия: <code>GET /api/reference/types-of-organization</code>

Метод	Путь	Описание	Ответ data
GET	/api/events/org/responsible/:id	Ответственные по мероприятию	Массив: [{ id_event, id_employees, first_name, second_name }]
GET	/api/events/org/notifications-today/:id	Мероприятия сотрудника на сегодня	Массив: [{ id, name, dates_of_event, day_of_the_week }]
GET	/api/events/org/notifications-tomorrow/:id	То же на завтра	то же

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/events/org/list	Список с фильтрами и пагинацией	filters — см. Org list and count filters , в т.ч. type (id типа организации). Также: sort, page, limit	{ success: true, data: [...], page, limit }
POST	/api/events/org/count	Количество по фильтрам	filters — те же, что в org/list (включая type)	{ success: true, total }
POST	/api/events/org	Создать мероприятие	name, form_of_holding, dates_of_event, day_of_the_week, amount_of_applications, amount_of_planning_application, annotation, result; опц.: type, link	{ success: true, data: { id } } (201)
POST	/api/events/org/responsible	Добавить ответственного	id_employee / id_employees, id_event	{ success: true, data: { ok: true } } (201)

Ошибки при создании мероприятия (POST /api/events/org)

Ответы об ошибках — JSON с success: false и полем error (текст), если не указано иное.

Код	Ситуация
201	Успех: { success: true, data: { id } } — id созданного мероприятия.
400	Не передан хотя бы один обязательный ключ из списка полей создания — сообщение валидации на русском.
400	Пустое или состоящее только из пробелов поле name .
500	Ошибка выполнения INSERT (БД недоступна, нарушение внешнего ключа: например type ссылается на несуществующий id в types_of_organization, и т.п.). Текст в error обычно общий («Не удалось создать мероприятие.»); детали смотрите в логах сервера.

Для PUT /api/events/org (обновление): те же **400** по обязательным полям и **name**; **404**, если мероприятие с указанным id не найдено; **500** при прочих ошибках БД.

Каждый элемент массива data в ответе /api/events/org/list: id, name, dates_of_event, day_of_the_week, type (число или null — id типа из types_of_organization; удобно проверять, что фильтр filters.type сработал).

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	/api/events/org/notifications	Мероприятия сотрудника на дату	id_employee , date (YYYY-MM-DD)	{ success: true, data: [...] }
PUT	/api/events/org	Обновить мероприятие	поля создания + id ; опц.: type , link	{ success: true, data: { ok: true } }

DELETE-эндпоинты

Метод	Путь	Описание	Тело / Параметры	Ответ
DELETE	/api/events/org/responsible	Удалить связь ответственный–мероприятие	Тело: id_employee , id_event	{ success: true, data: { ok: true } }
DELETE	/api/events/org/:id	Удалить мероприятие	Параметр пути: id	то же

Org list and count filters

В объект `filters` входят:

Поле	Описание
period	this_month , this_week , next_week , three_months , all — фильтр по полю <code>dates_of_event</code> . Если задан кастомный диапазон — <code>period</code> не используется.
date_from , date_to	YYYY-MM-DD — границы периода (включительно). Достаточно одного из полей. При наличии любого из них игнорируется <code>period</code> .
search	Подстрока в названии мероприятия (LIKE).
employee_id	Только мероприятия, где сотрудник с этим <code>id</code> является ответственным.
type	Положительное число — <code>id</code> типа из справочника <code>types_of_organization</code> (поле <code>type</code> в мероприятии). Список: GET /api/reference/types-of-organization . Некорректное значение игнорируется (фильтр по типу не применяется).

Сортировка (sort): один элемент { "field", "order" } . Допустимые поля: `id` , `name` , `dates_of_event` , `day_of_the_week` . `order` : `asc` или `desc` .

`page` по умолчанию 1, `limit` — от 1 до 100 (по умолчанию 10).

Документы к мероприятиям организации

Файлы, прикреплённые к мероприятию организации: метаданные в таблице `event_organization_document` , сами файлы — на диске под корнем `EVENT_DOCUMENTS_ROOT_ORG` (задаётся в настройках API приложения). В БД хранится только **относительный** путь `storage_path` (без `..`), совместимый с переносом сервера.

Метод	Путь	Описание
GET	/api/events/org/:eventId/documents	Список документов мероприятия
POST	/api/events/org/:eventId/documents	Загрузить файл
GET	/api/events/org/documents/:documentId/download	Скачать файл
PATCH	/api/events/org/documents/:documentId	Изменить порядок сортировки
DELETE	/api/events/org/documents/:documentId	Удалить документ (запись в БД + файл на диске)

Параметры пути

Параметр	Описание
eventId	Положительное целое — id мероприятия в event_plan_organization .
documentId	Положительное целое — id строки в event_organization_document .

GET /api/events/org/:eventId/documents

Возвращает массив метаданных без бинарного содержимого файла.

Ответ 200: { success: true, data: [...] }. Каждый элемент:

Поле	Описание
id	Id записи документа
id_event	Id мероприятия
storage_path	Относительный путь внутри корня EVENT_DOCUMENTS_ROOT_ORG
original_filename	Исходное имя файла
mime_type	MIME-тип или null
size_bytes	Размер в байтах или null
uploaded_by_profile_id	Id профиля пользователя, загрузившего файл, или null
sort_order	Порядок отображения (неотрицательное целое)
created_at	Дата/время создания записи

Ошибки: 400 — неверный eventId ; 404 — мероприятие не найдено; 503 — корневой каталог документов не задан в конфигурации API.

POST /api/events/org/:eventId/documents

Загрузка одного файла. Тело запроса — **multipart/form-data** , поле файла должно называться **file** (один файл за запрос).

Ответ 201: { success: true, data: { id, id_event, storage_path, original_filename, mime_type, size_bytes, uploaded_by_profile_id, sort_order } } — метаданные сохранённого документа; sort_order назначается автоматически (следующий после максимального для этого мероприятия).

Ограничения: максимальный размер файла — **50 МБ**; при превышении — 400 с текстом об ошибке.

Ошибки: 400 — нет файла в поле file или неверный eventId ; 404 — мероприятие не найдено; 503 — корень не настроен; 500 — ошибка записи на диск или в БД.

GET /api/events/org/documents/:documentId/download

Отдаёт файл как вложение (Content-Disposition: attachment , имя в кодировке UTF-8). Тело ответа — **бинарные данные файла**, не JSON. Заголовок Content-Type — из сохранённого mime_type или application/octet-stream .

Ошибки: 400 — неверный id; 404 — запись не найдена или файл отсутствует на диске; 503 — корень не настроен.

PATCH /api/events/org/documents/:documentId

Тело (JSON): { "sort_order": <неотрицательное целое> } — новое значение порядка отображения среди документов этого мероприятия (сортировка списка: sort_order по возрастанию, затем id).

Ответ 200: { success: true, data: { ok: true } } .

Ошибки: 400 — неверные параметры; 404 — документ не найден.

DELETE `/api/events/org/documents/:documentId`

Удаляет строку в БД и пытается удалить файл на диске; если файл уже отсутствует, операция всё равно считается успешной после удаления записи.

Ответ 200: `{ success: true, data: { ok: true } }`.

Ошибки: 400, 404, 503 — по смыслу как выше.

6. Events (part) — Мероприятия участия

Базовый путь: `/api/events/part`

Токен требуется.

Мероприятия типа «участие»: список с фильтрами и пагинацией, полная информация, ответственные, уведомления, CRUD, результат и отметка ответственного.

Фильтр списка и count по типу мероприятия (part)

Для `POST /api/events/part/list` и `POST /api/events/part/count` в `filters` укажите `id_type` (или то же значение в поле `type_id` — синоним): положительное целое — id из таблицы `type_of_part_event` (как при создании мероприятия). Справочник: `GET /api/reference/levels`. Поле `type` для org здесь **не** используется.

Пример:

```
"filters": {
  "id_type": 3,
  "search": "олимпиада"
}
```

Полный перечень полей — [Part list and count filters](#).

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	<code>/api/events/part/resp-table</code>	Таблица ответственных	Массив (<code>responsible_for_part_events</code>)
GET	<code>/api/events/part/full-inf/:id</code>	Полная информация о мероприятии участия	Объект: <code>id</code> , <code>name</code> , <code>form_of_holding</code> (число — id из <code>form_of_holding</code>), <code>id_type</code> (число — id из <code>type_of_part_event</code>), <code>registration_deadline</code> , <code>participants_and_works</code> , <code>result</code> , <code>dates_of_event</code> , <code>annotation</code> , <code>link</code> , <code>participants_amount</code> , <code>winner_amount</code> , <code>runner_up_amount</code> . Текстовые названия формата и уровня не возвращаются; справочники: <code>GET /api/reference/types-of-holding</code> , <code>GET /api/reference/levels</code>
GET	<code>/api/events/part/responsible/:id</code>	Ответственные по мероприятию	Массив: <code>[{ id_event, id_employees, first_name, second_name, mark_of_sending_an_application }]</code>
GET	<code>/api/events/part/responsible-new/:id</code>	То же + <code>result_of_responsible</code>	Массив: то же + поле <code>result_of_responsible</code>

Метод	Путь	Описание	Ответ data
GET	/api/events/part/notifications-today/:id	Мероприятия сотрудника, дедлайн сегодня	Массив
GET	/api/events/part/notifications-tomorrow/:id	То же на завтра	то же

В эндпоинтах `notifications-today`, `notifications-tomorrow`, `PUT .../notifications` в ответе поле `form_of_holding` возвращается как **название** (join со справочником), не как `id`.

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/events/part/list	Список с фильтрами и пагинацией	<code>filters</code> — см. Part list and count filters , в т.ч. <code>id_type</code> или <code>type_id</code> . Также: <code>sort</code> , <code>page</code> , <code>limit</code>	{ success: true, data: [...], page, limit }
POST	/api/events/part/count	Количество по фильтрам	<code>filters</code> — те же, что в <code>part/list</code> (включая <code>id_type</code> / <code>type_id</code>)	{ success: true, total }
POST	/api/events/part	Создать мероприятие участия	<code>name</code> , <code>form_of_holding</code> (число — <code>id</code> из <code>form_of_holding</code>), <code>id_type</code> (число — <code>id</code> из <code>type_of_part_event</code>), <code>registration_deadline</code> , <code>participants_and_works</code> , <code>annotation</code> , <code>dates_of_event</code> , <code>link</code> , <code>participants_amount</code> , <code>winner_amount</code> , <code>runner_up_amount</code> , <code>result</code> (опц.)	{ success: true, data: { id } } (201)
POST	/api/events/part/responsible	Добавить ответственного	<code>id_employee</code> / <code>id_employees</code> , <code>id_event</code>	{ success: true, data: { ok: true } } (201)

Ошибки при создании мероприятия участия (POST /api/events/part)

Код	Ситуация
201	Успех: { success: true, data: { id } }.
400	Не передан хотя бы один обязательный ключ из списка полей создания — сообщение валидации на русском.
400	Пустое или только пробелы в <code>name</code> .
400	<code>id_type</code> не является положительным целым числом.
400	Если <code>form_of_holding</code> передан не пустой строкой и не <code>null</code> , он должен быть положительным целым (<code>id</code> из справочника <code>form_of_holding</code>); иначе текст «Некорректный <code>form_of_holding</code> .».
500	Ошибка INSERT (БД, нарушение внешнего ключа на <code>id_type</code> / <code>form_of_holding</code> и т.д.). В ответе обычно «Не удалось создать мероприятие.»; детали — в логах сервера.

Для `PUT /api/events/part`: те же **400**; **404**, если мероприятие с `id` не найдено; **500** при прочих ошибках БД.

Каждый элемент `data` в ответе `/api/events/part/list`: `id`, `name`, `registration_deadline`, `participants_amount`, `result`, `id_type` (`id` из `type_of_part_event` — удобно проверять фильтр по типу).

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	<code>/api/events/part/notifications</code>	Мероприятия сотрудника с дедлайном на дату	<code>id_employee</code> , <code>date</code> (YYYY-MM-DD)	<code>{ success: true, data: [...] }</code>
PUT	<code>/api/events/part</code>	Обновить мероприятие	поля создания + <code>id</code>	<code>{ success: true, data: { ok: true } }</code>
PUT	<code>/api/events/part/result</code>	Обновить результат ответственного	<code>id_event</code> , <code>id_employee</code> / <code>id_employees</code> , <code>result_of_responsible</code> (опц.)	то же
PUT	<code>/api/events/part/mark</code>	Обновить отметку отправки заявки	<code>id_event</code> , <code>id_employee</code> / <code>id_employees</code> , <code>mark_of_sending_an_application</code> (0 или 1)	то же

DELETE-эндпоинты

Метод	Путь	Описание	Тело / Параметры	Ответ
DELETE	<code>/api/events/part/responsible</code>	Удалить ответственного	Тело: <code>id_employee</code> , <code>id_event</code>	<code>{ success: true, data: { ok: true } }</code>
DELETE	<code>/api/events/part/:id</code>	Удалить мероприятие	Параметр пути: <code>id</code>	то же

Part list and count filters

Те же правила, что для `org` (см. выше), но фильтрация по датам идёт по полю `registration_deadline`. Поле `type` для `org` здесь **не** используется; вместо него — `id_type` (см. ниже).

Поле	Описание
<code>id_type</code>	Положительное число — <code>id</code> уровня/типа из <code>type_of_part_event</code> (как в теле создания мероприятия). Список: <code>GET /api/reference/levels</code> . Некорректное значение игнорируется.
<code>type_id</code>	То же, что <code>id_type</code> (синоним для совместимости).

Сортировка: допустимые поля: `id`, `name`, `registration_deadline`, `participants_amount`, `dates_of_event`.

Документы к мероприятиям участия

Те же правила, что для [документов организации](#), но привязка к мероприятию участия (`event_plan_participation`) и корень на диске — `EVENT_DOCUMENTS_ROOT_PART`, таблица метаданных — `event_participation_document`.

Метод	Путь
GET	<code>/api/events/part/:eventId/documents</code>
POST	<code>/api/events/part/:eventId/documents</code>

Метод	Путь
GET	/api/events/part/documents/:documentId/download
PATCH	/api/events/part/documents/:documentId
DELETE	/api/events/part/documents/:documentId

Параметр `eventId` — id строки в `event_plan_participation`. Поля ответов и коды ошибок — аналогичны разделу `org`; отличие только в таблицах БД и переменной окружения с корневым каталогом.

Ученики мероприятий участия

Привязка учеников (`students`) к мероприятиям участия (`event_plan_participation`) с указанием статуса.

Таблицы:

- `event_part_student_status` — справочник статусов: Участник (1), Победитель (2), Призёр (3).
- `event_part_student` — связь мероприятие–ученик–статус. Один ученик может быть привязан к одному мероприятию только один раз (уникальный ключ `id_event` + `id_student`).

Список статусов доступен через справочник: GET `/api/reference/student-statuses`.

Метод	Путь	Описание
GET	/api/events/part/:eventId/students	Список учеников мероприятия
POST	/api/events/part/:eventId/students	Добавить ученика к мероприятию
PUT	/api/events/part/students/:id	Обновить статус ученика
DELETE	/api/events/part/students/:id	Удалить ученика из мероприятия

GET `/api/events/part/:eventId/students`

Возвращает массив учеников, привязанных к мероприятию.

Ответ:

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "id_student": 42,
      "id_status": 1,
      "surname": "Иванов",
      "name": "Пётр",
      "patronymic": "Сергеевич",
      "status_name": "Участник"
    }
  ]
}
```

POST `/api/events/part/:eventId/students`

Добавляет ученика к мероприятию.

Тело запроса (`application/json`):

Поле	Тип	Обязательное	Описание
<code>id_student</code>	number	да	id ученика из таблицы <code>students</code>

Поле	Тип	Обязательное	Описание
id_status	number	да	id статуса из event_part_student_status

Ответ (201):

```
{ "success": true, "data": { "id": 5 } }
```

Ошибки:

Код	Описание
400	Не указан id_student или id_status ; несуществующий FK
409	Ученик уже добавлен к этому мероприятию

PUT /api/events/part/students/:id

Обновляет статус привязки (параметр :id — id строки event_part_student).

Тело запроса (application/json):

Поле	Тип	Обязательное	Описание
id_status	number	да	Новый id статуса

Ответ:

```
{ "success": true, "data": { "ok": true } }
```

DELETE /api/events/part/students/:id

Удаляет привязку ученика к мероприятию (параметр :id — id строки event_part_student).

Ответ:

```
{ "success": true, "data": { "ok": true } }
```

7. Schedule — Расписание

Базовый путь: /api/schedule

Токен требуется.

Расписание занятий: общее, по преподавателю/группе/комнате/дате, CRUD занятий.

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	/api/schedule	Полное расписание	Массив: id, room, group, startTime, endTime, day, id_employees
GET	/api/schedule/teachers	Список преподавателей	Массив: [{ id, name }]
GET	/api/schedule/groups	Список групп	Массив: [{ id, name }]
GET	/api/schedule/by-teacher/:id	Расписание по преподавателю	Массив (занятия)

Метод	Путь	Описание	Ответ data
GET	/api/schedule/by-group/:id	Расписание по группе	то же
GET	/api/schedule/by-room/:id	Расписание по комнате	то же

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/schedule/by-date	Занятия по дате и комнате	date (YYYY-MM-DD), room_id	{ success: true, data: [...] }
POST	/api/schedule	Добавить занятие	room_id, group_id, start_time, end_time, day (1–7), employee_id	{ success: true, data: { id } } (201)

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	/api/schedule	Обновить занятие	id, room_id, group_id, start_time, end_time	{ success: true, data: { ok: true } }

DELETE-эндпоинты

Метод	Путь	Описание	Параметры пути	Ответ
DELETE	/api/schedule/:id	Удалить занятие	id — id занятия	{ success: true, data: { ok: true } }

8. Reference — Справочники

Базовый путь: /api/reference

Токен требуется.

Справочники: комнаты, уровни доступа, должности, документы, форматы проведения, типы мероприятий организации, уровни мероприятий (part), статусы учеников.

Метод	Путь	Описание	Ответ data
GET	/api/reference/rooms	Список комнат	Массив: [{ id, name, number }]
POST	/api/reference/rooms	Создать комнату	{ success, data: { id } } (201) — только уровни 1, 4, 6
PUT	/api/reference/rooms/:id	Обновить комнату	{ success, data: { ok: true } } — 1, 4, 6
DELETE	/api/reference/rooms/:id	Удалить комнату	{ success, data: { ok: true } } — 1, 4, 6; 409 если есть расписание/аренда
GET	/api/reference/access	Уровни доступа (все)	Массив: [{ id, name }]
GET	/api/reference/access/:employeeId	Уровень доступа сотрудника	Объект: { access_level_id } — 404 если профиль не найден
GET	/api/reference/positions	Должности (все)	Массив: [{ id, name }]
POST	/api/reference/positions	Создать должность	Объект: { id } (статус 201)
PUT	/api/reference/positions/:id	Обновить должность	Объект: { ok: true }

Метод	Путь	Описание	Ответ data
DELETE	/api/reference/positions/:id	Удалить должность	Объект: { ok: true } — 409 если есть активные сотрудники
GET	/api/reference/docs	Документы (справочник ссылок)	Массив: [{ id, name, link }]
POST	/api/reference/docs	Добавить документ	{ success, data: { id } } (201) — 1, 4, 6
PUT	/api/reference/docs/:id	Обновить документ	{ success, data: { ok: true } } — 1, 4, 6
DELETE	/api/reference/docs/:id	Удалить документ	{ success, data: { ok: true } } — 1, 4, 6
GET	/api/reference/types-of-holding	Форматы проведения	Массив строк таблицы form_of_holding (поля как в БД)
GET	/api/reference/types-of-organization	Типы мероприятий организации	Массив: [{ id, name }] — id соответствует id_type в таблице types_of_organization, поле type в event_plan_organization
GET	/api/reference/levels	Уровни мероприятий участия	Массив: [{ id, name }] — поле name соответствует type в таблице type_of_part_event
GET	/api/reference/student-statuses	Статусы учеников мероприятий участия	Массив: [{ id, name }] — (1 Участник, 2 Победитель, 3 Призёр)

Управление должностями (positions)

POST /api/reference/positions

Создаёт новую должность.

Тело запроса (application/json):

Поле	Тип	Обязательное	Описание
name	string	да	Название должности (до 150 символов)

Ответ (201):

```
{ "success": true, "data": { "id": 13 } }
```

PUT /api/reference/positions/:id

Обновляет должность.

Тело запроса (application/json):

Поле	Тип	Обязательное	Описание
name	string	да	Новое название

Ответ:

```
{ "success": true, "data": { "ok": true } }
```


DELETE /api/reference/positions/:id

Удаляет должность. Если к должности привязаны **активные** сотрудники (`is_active = 1`) — возвращает **409**. У **неактивных** сотрудников (`is_active = 0`) с этой должностью поле `position` автоматически устанавливается в `NULL` , после чего должность удаляется.

Ошибки:

Код	Описание
404	Должность не найдена
409	К должности привязаны активные сотрудники; сначала переназначьте их

Управление комнатами (rooms)

Доступ на запись: `access_level_id` **1, 4 или 6** (root, руководитель, администратор).

POST /api/reference/rooms

Тело запроса:

Поле	Тип	Обязательное	Описание
<code>name</code>	string	да	Название комнаты
<code>number</code>	string	нет	Номер/обозначение

Ответ (201): { "success": true, "data": { "id": 3 } }

PUT /api/reference/rooms/:id

Тело: `name` (обяз.), `number` (опц.). **Ответ:** { "success": true, "data": { "ok": true } }

DELETE /api/reference/rooms/:id

409 — комната используется в расписании или аренде.

Управление документами (docs)

POST /api/reference/docs

Поле	Тип	Обязательное
<code>name</code>	string	да
<code>link</code>	string	да

Ответ (201): { "success": true, "data": { "id": 1 } }

PUT /api/reference/docs/:id / DELETE /api/reference/docs/:id

Аналогично `positions`; при удалении — 404 если не найден.

9. Rent — Аренда помещений

Базовый путь: `/api/rent`

Токен требуется.

Аренда помещений: по мероприятию, по id, по дате и комнате, создание, обновление, удаление.

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	<code>/api/rent/by-event/:id</code>	Список аренд по мероприятию	Массив
GET	<code>/api/rent/by-id/:id</code>	Одна запись аренды	Объект: { id_rent, id_event, id_room, start_time, end_time, date, room_name }

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	<code>/api/rent/by-date-room</code>	Аренды на дату и комнату	date (YYYY-MM-DD), room_id	{ success: true, data: [...] }
POST	<code>/api/rent</code>	Создать аренду	event_id, room_id, date, start_time, end_time	{ success: true, data: { id } } (201)

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	<code>/api/rent</code>	Обновить аренду	id, event_id, room_id, date, start_time, end_time	{ success: true, data: { ok: true } }

DELETE-эндпоинты

Метод	Путь	Описание	Параметры пути	Ответ
DELETE	<code>/api/rent/:id</code>	Удалить аренду	id — id аренды	{ success: true, data: { ok: true } }

10. Students — Студенты

Базовый путь: `/api/students`

Токен требуется.

Студенты: поиск по букве, по группе, по id, группы студента, проверка существования, добавление/обновление, связь с группами.

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	<code>/api/students/search</code>	Поиск по фамилии (буква)	Массив (пустая буква — все)
GET	<code>/api/students/search/:letter</code>	То же, буква в пути	то же
GET	<code>/api/students/search-new</code>	Поиск с isActive, кол-во групп	Массив

Метод	Путь	Описание	Ответ data
GET	/api/students/search-new/:letter	То же с letter	то же
GET	/api/students/groups-by-student/:id	Группы студента	Массив
GET	/api/students/by-group/:id	Краткий список студентов группы	Массив: [{ id, name }]
GET	/api/students/full-by-group/:id	Полная информация по студентам группы	Массив (все поля)
GET	/api/students/:id	Полная информация о студенте	Объект (один студент)

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/students/add-to-group	Добавить студента в группу	student_id, group_id	{ success: true, data: { ok: true } } (201)
POST	/api/students	Создать студента	surname, name, patronymic, birthDay, parentSurname, parentName, parentPatronymic, email, phone; опц.: navigator	{ success: true, data: { id } } (201)

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	/api/students/exist	Проверить существование по ФИО	surname, name, patronymic (string)	{ success: true, data: { amount } }
PUT	/api/students	Обновить студента	id + поля как при создании	{ success: true, data: { ok: true } }
PUT	/api/students/update-to-group	Перевести из группы в группу	student_id, old_group_id, new_group_id	то же

DELETE-эндпоинты

Метод	Путь	Описание	Тело / Query	Ответ
DELETE	/api/students/from-group	Удалить студента из группы	student_id, group_id	{ success: true, data: { ok: true } }

11. Attendance — Посещаемость

Базовый путь: /api/attendance

Токен требуется.

Посещаемость: по группе, по группе и дате, создание/обновление записи.

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	/api/attendance/by-group/:id	Посещаемость по группе (по дате DESC)	Массив: name, date_of_lesson, presence

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	/api/attendance/by-group-date	Посещаемость по группе и дате	group_id, date (дата урока)	{ success: true, data: [...] } — name, date_of_lesson, presence
PUT	/api/attendance/by-group-date-new	То же с id_student в каждом элементе	то же	то же + id_student

POST-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/attendance	Создать/обновить запись (upsert)	student_id, group_id, date_of_lesson (YYYY-MM-DD), presence (0/1)	{ success: true, data: { ok: true } } (201)
POST	/api/attendance/clear-all	Полная очистка таблицы attendance	—	см. ниже

POST /api/attendance/clear-all

Массовое удаление **всех** записей посещаемости. Только уровни доступа **1, 4, 6**. Транзакция, запись в audit-лог сервера.

Ответ 200 (без обёртки data):

```
{
  "success": true,
  "message": "Вся посещаемость очищена",
  "affectedRows": 42
}
```

Код	Описание
403	Недостаточно прав
500	Ошибка БД

12. Groups — Группы

Базовый путь: /api/groups

Токен требуется.

Группы: по преподавателю, таблица студент–группа, пиксели по группе, список групп, обновление пикселей.

GET-эндпоинты

Метод	Путь	Описание	Ответ data
GET	/api/groups/by-teacher/:id	Группы преподавателя	Массив
GET	/api/groups/table	Таблица students_groups	Массив (все записи)
GET	/api/groups/pixels/:id	Пиксели студентов по группе	Массив: name + поля pixels
GET	/api/groups/list	Список групп	Массив: [{ id, name }]

POST-эндпоинты (группы и пиксели)

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/groups/list	Создать группу	name (string)	{ success: true, data: { id } } (201); 409 — дубликат имени; 1, 4, 6
POST	/api/groups/pixels/clear-all	Обнулить все столбцы пикселей у всех учеников	—	см. ниже

POST /api/groups/pixels/clear-all

Только **1, 4, 6**. В транзакции все поля пикселей (part_of_comp , make_content , ...) устанавливаются в 0 .

Ответ 200:

```
{
  "success": true,
  "message": "Пиксели у всех учеников очищены",
  "affectedRows": 120
}
```

PUT-эндпоинты

Метод	Путь	Описание	Тело запроса	Ответ
PUT	/api/groups/pixels	Обновить пиксели студента	id_student или id; плюс поля: part_of_comp, make_content, invite_friend, clean_kvantum, filled_project_card_on_time, finished_project_with_product, regional_competition, interregional_competition, all_russian_competition, international_competition, nto, become_an_engineering_volunteer, help_with_event, make_own_event, special_achievements, fine (числа)	{ success: true, data: { ok: true, affected } }
PUT	/api/groups/list/:id	Переименовать группу	name	{ success: true, data: { ok: true } } ; 1, 4, 6

DELETE-эндпоинты

Метод	Путь	Описание	Ответ
DELETE	/api/groups/list/:id	Удалить группу	{ success: true, data: { ok: true } } ; 409 — есть ученики/расписание; 1, 4, 6

13. Desktop Updates — Обновления клиента

Базовый путь: /desktop-updates

Токен не требуется. Используется desktop-клиентом (electron-updater).

Каталог файлов обновлений задаётся переменными окружения DESKTOP_UPDATES_ROOT или DESKTOP_UPDATES_DIR . Если не заданы — <рабочая_директория_процесса>/desktop-updates . В каталоге должны лежать latest.yml , установщик .exe и соответствующий .exe.blockmap (имена из latest.yml).

Метод	Путь	Описание	Ответ
GET	/desktop-updates/latest.yml	Манифест обновления	Текст YAML
GET	/desktop-updates/:installerName.exe	Установщик Windows	Бинарный файл
GET	/desktop-updates/:installerName.exe.blockmap	Blockmap для дельта-обновления	Бинарный файл

Ошибки: 404 — файл не найден или latest.yml ссылается на отсутствующие артефакты.

14. Коды ошибок и пагинация

Коды HTTP

Код	Описание
200	Успех
201	Создано
400	Неверный запрос (валидация, неверные параметры)
401	Не авторизован (нет или неверный токен)
403	Доступ запрещён
404	Не найдено
409	Конфликт (например, дубликат)
500	Ошибка сервера
503	Сервис недоступен по конфигурации (например, не заданы корневые каталоги документов мероприятий)

Пагинация (list + count)

Для эндпоинтов с пагинацией (/api/events/org/list , /api/events/part/list):

1. Запросите **count** с теми же `filters` — получите `total`.
2. Вычислите `totalPages = ceil(total / limit)`.
3. Запрашивайте **list** с `page=1` , `page=2` , ... ПОКА `page <= totalPages`.

Подробности по `filters` (периоды, `date_from` / `date_to` , `search` , `employee_id` , фильтр по типу: `type` для `org`, `id_type` или `type_id` для `part`) — в подразделах [Org list and count filters](#) и [Part list and count filters](#).
Дополнительно: [API СОБЫТИЯ.md](#).

Пример тела запроса для list (org):

```
{
  "filters": {
    "period": "this_month",
    "search": "текст",
    "employee_id": 1,
    "type": 2
  },
  "sort": [
    { "field": "name", "order": "asc" }
  ],
  "page": 1,
  "limit": 20
}
```

Поле `type` в `filters` — необязательное; это id типа из `types_of_organization` (см. раздел «Фильтр списка и count по типу мероприятия (org)» выше).

Пример с кастомным диапазоном дат (part, по `registration_deadline`):

```
{
  "filters": {
    "date_from": "2025-01-01",
    "date_to": "2025-02-28",
    "id_type": 1
  },
  "sort": [{ "field": "registration_deadline", "order": "desc" }],
  "page": 1,
  "limit": 10
}
```

Поле `id_type` в `filters` — необязательное; id уровня из `type_of_part_event` (см. «Фильтр списка и count по типу мероприятия (part)» выше).

Ответ:

```
{
  "success": true,
  "data": [...],
  "page": 1,
  "limit": 20
}
```

Дополнительные материалы

- [РУКОВОДСТВО.md](#) — общее руководство по приложению
- [HEADLESS_FULL_MANUAL.md](#) — развёртывание headless-версии
- [API_СОБЫТИЯ.md](#) — мероприятия: фильтры list/count, кастомные даты, полный ответ `GET .../part/full-inf/:id`
- [EVENTS_DOCUMENTS_AND_CONTRIBUTIONS.md](#) — модель документов к мероприятиям и вклад ответственных (part)
- Сборка PDF/DOCX: `npm run doc:api` → `files/docs/API_DOCUMENTATION.pdf` и `.docx` (источник для PDF — этот файл; табличная сводка — `scripts/api-documentation-data.js`)